ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

NT106 — LẬP TRÌNH MẠNG CĂN BẢN

CẨM NANG LABGUARD

HƯỚNG DẪN THÀNH VIÊN A

AGENT & GIAO THỨC TRUYỀN THÔNG

Phân hệ máy trạm và chuẩn đóng gói dữ liệu mạng cho hệ thống LabGuard

BIÊN SOẠN: VÕ TRỌNG PHÚC

NHÓM THỰC HIỆN: 4 THÀNH VIÊN

TP. HỒ CHÍ MINH — 2026

KHỞI ĐỘNG NHANH DÀNH CHO THÀNH VIÊN A

(Đọc xong trang này trong 2 phút là bạn biết chính xác mình cần làm gì hôm nay)

1. BẠN LÀ AI?

Bạn là người phụ trách phần mềm chạy ngầm trên máy sinh viên (**Agent**) và thư viện giao thức đóng gói dữ liệu mạng (**Protocol**).

2. BẠN LÀM GÌ?

- Đọc hiểu thư viện giao thức `src/LabGuard.Protocol/` (đã hoàn thành 100%, bộ kiểm thử giao thức đã PASS).
- Lập trình Agent mở kết nối TCP tới máy chủ Central Server (cổng 5000).
- Định kỳ 5 giây đo tải CPU, RAM và gửi nhịp tim sống còn lên máy chủ.
- Giữ lại số liệu vào bộ nhớ đệm (Replay Buffer) khi bị đứt mạng và tự động gửi bù khi có mạng lại.

3. BẠN KHÔNG CẦN LO PHẦN NÀO?

- Bạn **không cần** lo việc ghi cơ sở dữ liệu SQL Server hay viết luật phát hiện sự cố (việc của **Thành viên B**).
- Bạn **không cần** lo vẽ giao diện đồ họa WinForms hay bảng điều khiển (việc của **Thành viên C**).
- Bạn **không cần** lo trạm trung chuyển Gateway hay công cụ giả lập kiểm thử tải (việc của **Thành viên D**).

4. HÔM NAY BẮT ĐẦU Ở ĐÂU?

Chạy lệnh `dotnet test tests/LabGuard.Agent.Tests` để thấy 16 bài kiểm thử A-003 màu xanh. Tầng kết nối mạng của Agent đã hoàn thành trong `AgentSocketClient.cs`. Bước tiếp theo của bạn là thực hiện chặng A-004: tạo `MetricsCollector.cs` để đo CPU, RAM, Uptime thật.

5. FILES BẠN CẦN MỞ TRƯỚC:

1. `START_HERE.md`: Bản đồ hướng dẫn khởi động nhanh dự án.
2. `src/LabGuard.Protocol/WireEnvelope.cs`: Chiếc phong bì chuẩn 6 trường của toàn hệ thống.
3. `src/LabGuard.Protocol/FrameCodec.cs`: Cách cắt và ghép 4 byte độ dài khi gửi tin qua mạng.
4. `src/LabGuard.Agent/Program.cs`: Nơi bạn sẽ viết mã chạy cho phần mềm Agent.

MỤC LỤC

1 Tổng quan vai trò của Thành viên A **1**

2 Chặng 1: Thư viện Giao thức (ĐÃ HOÀN THÀNH) **2**
2.1 Cấu trúc chuẩn của WireEnvelope.cs . . . 2
2.2 Bộ đóng khung nhị phân FrameCodec.cs . . . 3
2.3 Giải thuật đọc chính xác N byte (ReadExactlyBytesAsync) . . . 4
2.4 Bộ kiểm tra message đúng luật (EnvelopeValidator.cs) . . . 4
2.5 Bộ chuyển đổi JSON (ProtocolSerializer.cs) . . . 5
2.6 Cấu trúc bộ kiểm thử giao thức thực tế (Protocol Tests) . . . 5

3 Các chặng công việc máy trạm Agent **5**
3.1 ĐÃ HOÀN THÀNH (A-003): Kết nối TCP tới Central và hàng đợi gửi tin an toàn 6
3.2 Chặng 3: VIỆC TIẾP THEO: Đo số liệu tài nguyên phần cứng (CPU, RAM) . . . 7
3.3 Chặng 4: VIỆC TIẾP THEO: Gửi nhịp tim định kỳ duy trì sự sống (Heartbeat) . . 8
3.4 Chặng 5: VIỆC TIẾP THEO: Bộ đệm ngoại tuyến lưu tạm khi rớt mạng (Replay Buffer) . . . 10
3.5 Chặng 6: VIỆC TIẾP THEO: Giám sát tiến trình phần mềm quy định . . . 11
3.6 Chặng 7: VIỆC TIẾP THEO: Tự động hóa kiểm thử cho Agent . . . 12

4 Năm bẫy lỗi lập trình cần tránh dành cho Thành viên A **13**

5 Bộ câu hỏi vấn đáp dành riêng cho Thành viên A **15**

1. Tổng quan vai trò của Thành viên A

TÓM TẮT 30 GIÂY

- Bạn chịu trách nhiệm hai phần: thư viện giao thức chung (**Protocol**) và phần mềm máy trạm (**Agent**).
- Thư viện `Protocol` đã hoàn thành 100% (toàn bộ kiểm thử giao thức đã PASS), là hợp đồng mạng bất biến cho cả nhóm.
- Phần mềm `Agent` hiện đã có nền tảng kết nối TCP và hàng đợi gửi tuần tự của A-003. Các phần thu thập dữ liệu, heartbeat, reconnect, replay và giám sát dịch vụ vẫn chưa được triển khai.
- Bạn phối hợp chặt chẽ với Thành viên B (máy chủ tiếp nhận các gói tin của bạn).

Chào mừng bạn đến với phân hệ nền tảng của đồ án LabGuard. Trong nhóm 4 người, bạn là người đặt những viên gạch đầu tiên cho toàn bộ việc kết nối mạng của hệ thống:

KHÁI NIỆM MẠNG CỐT LÕI

Hai khu vực mã nguồn do Thành viên A phụ trách:

1. **Thư viện Giao thức dùng chung (`src/LabGuard.Protocol/`):** Định nghĩa cấu trúc phong bì dữ liệu `WireEnvelope` và bộ đóng khung `FrameCodec`. Đây là hợp đồng mạng bất biến mà cả 3 thành viên còn lại (B, C, D) đều phải dựa vào để trao đổi thông tin. Phân hệ này hiện **ĐÃ HOÀN THÀNH 100%** với bộ kiểm thử tự động đã PASS.
2. **Phần mềm Máy trạm Agent (`src/LabGuard.Agent/`):** Lập trình ứng dụng chạy ngầm trên từng máy tính sinh viên, định kỳ đo tài nguyên CPU/RAM, gửi nhịp tim sống còn và giữ lại dữ liệu khi bị đứt cáp mạng. **Tác vụ A-003 (kết nối TCP & hàng đợi Single-Writer) đã XONG (16/16 tests PASS).** Các tác vụ A-004+ là **CHƯA LÀM** để bạn tự tay xây dựng.

Hình này chỉ cần nhìn theo mũi tên từ dưới lên trên: Agent sử dụng thư viện Protocol chung của dự án.

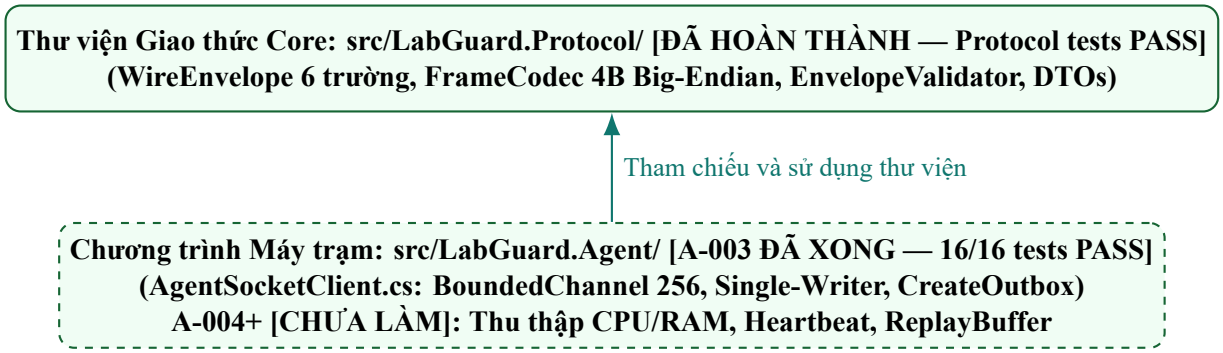


Figure 1: Sơ đồ mối quan hệ giữa 2 phân hệ do Thành viên A phụ trách

Ý chính: Thư viện Protocol đã hoàn thành 100% với bộ kiểm thử giao thức đã PASS, đóng vai trò nền móng bất biến để bạn yên tâm lập trình Agent.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Thư viện Protocol là tài sản chung của cả nhóm, tuyệt đối không tự ý đổi cấu trúc 6 trường phong bì.
- Việc tiếp theo của bạn tập trung 100% vào thư mục `src/LabGuard.Agent/`.

2. Chặng 1: Thư viện Giao thức & Đóng khung tin (ĐÃ HOÀN THÀNH)

TÓM TẮT 30 GIÂY

- Dữ liệu gửi qua mạng luôn bỏ vào phong bì `WireEnvelope` đúng 6 trường.
- Trước khi gửi qua socket TCP, thêm 4 byte độ dài Big-Endian (`FrameCodec`).
- Khi nhận dữ liệu, phải dùng vòng lặp gom đủ byte (`ReadExactlyBytesAsync`), không được đọc nửa vời.
- Đã có sẵn bộ kiểm thử tự động PASS chứng minh thư viện chạy đúng 100%.

GHI CHÚ QUAN TRỌNG

Đây là phần việc **ĐÃ HOÀN TẤT** trong mã nguồn thực tế. Toàn bộ 107 bài test tự động trong `tests/LabGuard.Protocol.Tests` đều đã chạy thành công mỹ mãn. Bạn cần đọc hiểu sâu sắc phần này để tự tin khi viết mã cho Agent và khi trả lời vấn đáp.

2.1 Cấu trúc chuẩn của WireEnvelope.cs

Mọi dữ liệu truyền qua mạng trong LabGuard đều được đặt vào một chiếc "phong bì" JSON có **CHÍNH XÁC 6 TRƯỜNG CỐ ĐỊNH**. Dưới đây là trích xuất từ file `src/LabGuard.Protocol/WireEnvelope`

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- 6 trường cố định JSON: `version`, `messageType`, `requestId`, `deviceId`, `sentAtUtc`, `payload`.
- Hàm tĩnh `Create()` giúp tự động điền Version = 1, sinh UUID ngẫu nhiên và lấy giờ UTC chuẩn.

```
// Trich doan tu ma nguon thuc te: 6 thuoc tinh phong bi du lieu chuan
public class WireEnvelope<T>
{
    [JsonPropertyName("version")]     public int Version { get; set; } = 1;
    [JsonPropertyName("messageType")] public string MessageType { get; set; } =
        string.Empty;
    [JsonPropertyName("requestId")]   public string RequestId { get; set; } =
        string.Empty;
    [JsonPropertyName("deviceId")]    public string DeviceId { get; set; } =
        string.Empty;
    [JsonPropertyName("sentAtUtc")]   public string SentAtUtc { get; set; } =
        string.Empty;
    [JsonPropertyName("payload")]     public T Payload { get; set; } = default!;
```

```
10
11      // Phuong thuc Create(...) tien ich khoi tao nhanh (xem file nguon day du)
12      public static WireEnvelope<T> Create(string type, string devId, T payload,
            string? reqId = null) => ...;
13  }
```

Listing 1: Ví dụ rút gọn dựa trên mã nguồn hiện tại: src/LabGuard.Protocol/WireEnvelope.cs

Ý nghĩa 6 trường phong bì:

- `version`: Số nguyên 1. Tuyệt đối không dùng dạng chuỗi `"1.0"`.
- `messageType`: Tên thông điệp viết hoa (ví dụ: `HEARTBEAT`, `METRIC_BATCH`).
- `requestId`: Mã UUID v4 duy nhất cho mỗi lượt gửi (36 ký tự).
- `deviceId`: Tên máy gửi bản tin (ví dụ: `PC-LAB1-01` hoặc `LAB01-PC02`).
- `sentAtUtc`: Thời điểm gửi theo giờ UTC quốc tế, kết thúc bằng Ź hoặc `+00:00`.
- `payload`: Dữ liệu chi tiết của từng loại thông điệp (tham số generic `T`, như `JsonElement` hoặc DTO cụ thể).

2.2 Bộ đóng khung nhị phân FrameCodec.cs

TCP là dòng byte liên tục (*Byte Stream*), không có khái niệm gói tin nguyên vẹn. Lớp `FrameCodec.cs` giải quyết vấn đề này bằng cách thêm 4 byte độ dài Big-Endian vào đầu mỗi chuỗi JSON trước khi truyền qua socket:

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Tiêu đề 4 byte Big-Endian chứa kích thước chính xác của mảng byte payload.
- Hạn mức an toàn `MaxPayloadSize = 262_144` (256 KiB) giúp bảo vệ hệ thống khỏi tràn bộ nhớ.

```
1  // Trich doan: Hang so co dinh 4 byte do dai va gioi han 256 KiB
2  public static class FrameCodec
3  {
4      public const int HeaderSize = 4;                                    // 4 byte do
           dai Big-Endian
5      public const int MaxPayloadSize = 262_144;                          // 256 KiB
6      public const int MaxWireFrameSize = HeaderSize + MaxPayloadSize; // 262,148
           bytes
7  }
```

Listing 2: Ví dụ rút gọn dựa trên mã nguồn hiện tại: Hằng số giới hạn trong FrameCodec.cs

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- `BinaryPrimitives.WriteUInt32BigEndian` ghi đúng thứ tự byte mạng chuẩn mực.
- Nối liền 4 byte độ dài với mảng byte nội dung JSON phía sau.

```
1  // Trich doan: Dong goi 4 byte Big-Endian vao dau mang byte
2  public static byte[] Encode(ReadOnlyMemory<byte> payload)
3  {
4      if (payload.Length == 0 || payload.Length > MaxPayloadSize)
5          throw new ArgumentException("Kich thuoc payload khong hop le.");
6
```

```
    byte[] frame = new byte[HeaderSize + payload.Length];
    BinaryPrimitives.WriteUInt32BigEndian(frame.AsSpan(0, HeaderSize),
        (uint)payload.Length);
    payload.Span.CopyTo(frame.AsSpan(HeaderSize));
    return frame;
}
```

Listing 3: Ví dụ rút gọn dựa trên mã nguồn hiện tại: Phương thức Encode trong FrameCodec.cs

2.3 Giải thuật đọc chính xác N byte (ReadExactlyBytesAsync)

Khi nhận dữ liệu từ mạng, một lần gọi `stream.ReadAsync` có thể chỉ trả về một vài byte đầu tiên do phân mảnh mạng. Phương thức `ReadExactlyBytesAsync` dùng vòng lặp tích lũy để gom đủ từng byte:

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Vòng lặp `while (totalRead < count)` tiếp tục đọc cho tới khi nhận đủ số byte yêu cầu.
- Nếu kết quả `read == 0`, nghĩa là đối phương đã chủ động ngắt kết nối mạng.

```
// Trich doan: Vong lap gom tich luy du count byte tu NetworkStream
public static async Task ReadExactlyBytesAsync(
    Stream stream, byte[] buffer, int offset, int count, CancellationToken ct =
        default)
{
    int totalRead = 0;
    while (totalRead < count)
    {
        int read = await stream.ReadAsync(buffer.AsMemory(offset + totalRead,
            count - totalRead), ct);
        if (read == 0)
            throw new EndOfStreamException($"Mat ket noi TCP: da doc
                {totalRead}/{count} bytes.");
        totalRead += read;
    }
}
```

Listing 4: Ví dụ rút gọn dựa trên mã nguồn hiện tại: ReadExactlyBytesAsync trong FrameCodec.cs

2.4 Bộ kiểm tra message đúng luật (EnvelopeValidator.cs)

Lớp `EnvelopeValidator.cs` thi hành nguyên tắc bảo mật: *"Không bao giờ tin tưởng dữ liệu thô từ mạng"*. Mỗi bản tin sau khi giải mã JSON đều được kiểm tra:

- Kiểm tra `Version == 1`.
- Kiểm tra `MessageType` không rỗng và thuộc danh mục 69 loại chuẩn trong `MessageTypes.cs`.
- Kiểm tra `RequestId` là chuỗi GUID 36 ký tự hợp lệ.
- Kiểm tra `DeviceId` không được rỗng và không chứa ký tự nguy hiểm.
- Kiểm tra `SentAtUtc` có chỉ định múi giờ UTC tường minh (Ź hoặc `+00:00`).

2.5 Bộ chuyển đổi JSON (ProtocolSerializer.cs)

Lớp `ProtocolSerializer.cs` cấu hình thư viện `System.Text.Json` với các tùy chọn chuẩn hóa giúp chuỗi JSON luôn gọn gàng (minified) và không tốn byte thừa trên đường truyền.

2.6 Cấu trúc bộ kiểm thử giao thức thực tế (Protocol Tests)

Thư viện kiểm thử `tests/LabGuard.Protocol.Tests` bao gồm 107 bài test tự động:

1. **`FrameCodecTests.cs`:** Kiểm thử đóng khung 4 byte Big-Endian, đọc phân mảnh TCP, phát hiện khung quá khổ `MaxPayloadSize` (256 KiB).
2. **`ProtocolSpecificationTests.cs`:** Kiểm thử tuân thủ đặc tả giao thức, kiểm tra 6 trường outer, xác thực giờ UTC.
3. **`WireContractAndFixturesTests.cs`:** Kiểm thử khớp 16 fixture JSON quy chuẩn và tuần tự hóa hai chiều.
4. **`ChunkedStream.cs`:** Luồng giả lập ngắt quãng từng byte để thử nghiệm khả năng chịu phân mảnh mạng.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Thư viện Protocol đã pass toàn bộ 107/107 tests tự động.
- Khi làm Agent, bạn chỉ việc gọi `FrameCodec.Encode(env)` để gửi và `FrameCodec.ReadFrameAsync(stream)` để nhận.

3. Chặng 2 — A-003 ĐÃ XONG & Chặng 3–7 (Chưa làm / Dự kiến)

TÓM TẮT 30 GIÂY

- Agent chạy ngầm trên máy sinh viên, kết nối TCP tới Central Server (Port 5000).
- Dùng hàng đợi một luồng ghi (`Single-Writer Channel`) để gửi tin an toàn không bị trộn byte (A-003 ĐÃ XONG).
- A-004 sẽ bổ sung HEARTBEAT và METRIC_BATCH định kỳ.
- A-006 dự kiến bổ sung ReplayBuffer tối đa 50 METRIC_BATCH.

GHI CHÚ QUAN TRỌNG

Chặng 2 (A-003) ĐÃ HOÀN THÀNH: `AgentSocketClient.cs` với BoundedChannel 256, Single-Writer Loop, `CreateOutbox` factory, xử lý lỗi ghi mạng và DisposeAsync đúng — 16/16 kiểm thử PASS.
Các chặng từ 3 đến 7 dưới đây đang ở trạng thái **CHƯA LÀM / DỰ KIẾN**. Bạn sẽ tự tay lập trình từng lớp đối tượng trong thư mục `src/LabGuard.Agent/`.

3.1 ĐÃ HOÀN THÀNH (A-003): Kết nối TCP tới Central và hàng đợi gửi tin an toàn

Chặng 2 — Mã tác vụ nội bộ: LG-A-003 — XONG (16/16 tests PASS)

3.1.1 Tại sao cần phần này?

Trong một chương trình Agent, có hai việc xảy ra đồng thời: một luồng định kỳ gửi nhịp tim sống còn (`HEARTBEAT` mỗi 5s), một luồng khác định kỳ gửi số liệu CPU/RAM (`METRIC_BATCH` mỗi 5s). Nếu cả hai luồng cùng gọi thẳng hàm ghi vào socket TCP (`stream.WriteAsync`), các byte của hai gói tin sẽ bị trộn lẫn vào nhau, khiến máy chủ không thể đọc được và ngắt kết nối ngay!

3.1.2 Khi chạy, chuyện gì xảy ra?

Agent khởi tạo `TcpClient` kết nối đến Central Server trên cổng 5000. Để đảm bảo an toàn tuyệt đối mà không cần dùng khóa chặn nghẽn (`lock`), ta dùng một hàng đợi có giới hạn dung lượng (**Bounded Single-Writer Queue**) bằng `Channel<byte[]>`:

- Các luồng nghiệp vụ (Heartbeat, đo CPU/RAM) chỉ việc đẩy mảng byte vào hàng đợi.
- Duy nhất một luồng nền độc lập lấy từng gói tin ra và ghi tuần tự vào `NetworkStream`.

Hình này chỉ cần nhìn theo các mũi tên chuyển đổi trạng thái: bắt đầu từ Khởi động đến Đang kết nối và Chờ thử lại.

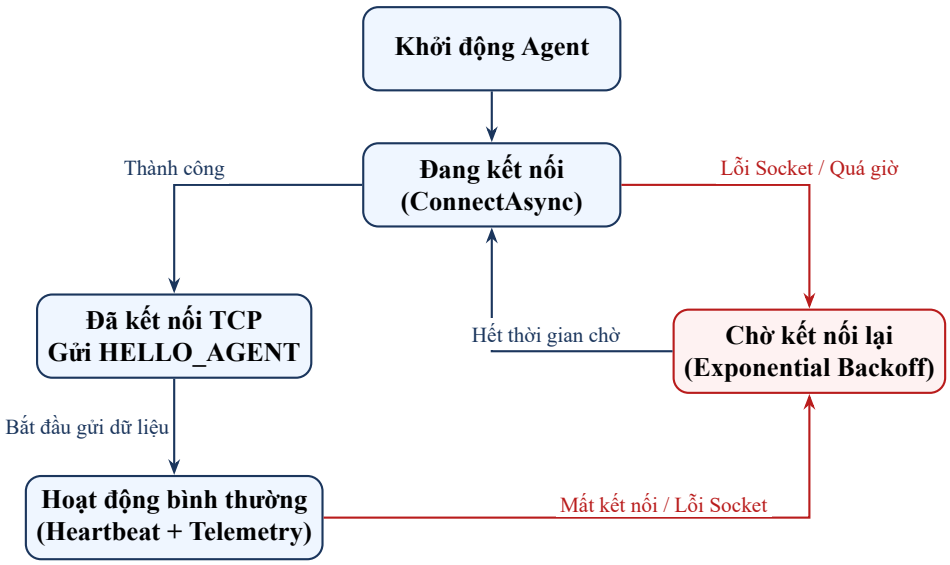


Figure 2: Sơ đồ máy trạng thái kết nối và tự động kết nối lại của Agent

Ý chính: Khi mất mạng, Agent không được sập chương trình mà phải chuyển sang trạng thái chờ thử lại với thời gian tăng dần (Exponential Backoff).

3.1.3 Bạn sẽ đụng file nào?

Tạo mới file: `src/LabGuard.Agent/AgentSocketClient.cs`.

3.1.4 Bạn cần làm những bước lớn nào?

1. Mở kết nối TCP tới máy chủ bằng `TcpClient.ConnectAsync(host, port)`.

2. Khởi tạo hàng đợi có giới hạn `BoundedChannelOptions(256)` với chế độ chờ `FullMode.Wait` để bộ nhớ không tăng vô hạn.
3. Khởi chạy vòng lặp gửi tin đơn lẻ (Single Writer Loop) đọc từ Channel và ghi vào socket.
4. Chạy vòng lặp nhận tin ngầm đọc từ socket bằng `FrameCodec.ReadExactlyBytesAsync`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Kết nối tới máy chủ và duy trì luồng mạng `NetworkStream`.
- Đọc từng khung tin nhị phân bằng `FrameCodec` mà không làm sập chương trình khi đứt mạng.

```
// Pseudocode minh hoa: Ket noi va doc khung tin tu Central
public async Task RunClientAsync(string host, int port, CancellationToken ct)
{
    using var tcp = new TcpClient();
    await tcp.ConnectAsync(host, port, ct);
    using var stream = tcp.GetStream();
    while (!ct.IsCancellationRequested)
    {
        byte[]? frame = await FrameCodec.ReadFrameAsync(stream, ct: ct);
        if (frame == null) break; // Central dong ket noi
        ProcessIncomingFrame(frame);
    }
}
```

Listing 5: Pseudocode minh họa: Vòng lặp kết nối và nhận tin của AgentSocketClient

3.1.5 Làm sao thử?

Bật Agent lên khi Central chưa bật: Agent phải in log đang chờ kết nối lại (1s, 2s, 4s . . . 30s) mà không bị văng. Sau đó bật Central lên: Agent lập tức phát hiện và kết nối thành công.

3.1.6 Khi nào coi là xong? (PASS Criteria)

- Kết nối thành công tới Central Server trên cổng 5000.
- Tự động thử kết nối lại với thuật toán Exponential Backoff (trần 30s) khi mạng gián đoạn.
- Đọc trọn vẹn từng khung tin nhị phân mà không bị vỡ gói.

3.2 Chặng 3: VIỆC TIẾP THEO: Đo số liệu tài nguyên phần cứng (CPU, RAM)

3.2.1 Tại sao cần phần này?

Mục đích chính của LabGuard là giám sát xem máy sinh viên có bị treo, đơ hay quá tải không. Ta phải đo được tỷ lệ phần trăm CPU và dung lượng RAM còn trống của máy để báo về trung tâm.

3.2.2 Khi chạy, chuyện gì xảy ra?

Định kỳ mỗi 5 giây, một tác vụ nền đọc các giá trị hiệu năng từ hệ điều hành Windows:

- Mức tải CPU hiện tại (ví dụ: 35.4%).

- Dung lượng RAM khả dụng (ví dụ: 4096 MB/8192 MB).

Agent đóng gói các con số này vào `WireEnvelope` với loại thông điệp `METRIC_BATCH` và gửi qua socket về Central Server.

3.2.3 Bạn sẽ đụng file nào?

Tạo mới file: `src/LabGuard.Agent/MetricsCollector.cs`.

3.2.4 Bạn cần làm những bước lớn nào?

1. Dùng `Process` hoặc Performance Counter để lấy tỷ lệ CPU.
2. Dùng `GC.GetGCMemoryInfo()` hoặc Windows API để lấy dung lượng RAM còn trống.
3. Đóng gói kết quả thành danh sách `MetricSampleDto`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Hàm `CollectCpu()` trả về số phần trăm tải CPU từ 0 đến 100.
- Hàm `CollectRam()` tính dung lượng bộ nhớ khả dụng theo Megabyte (MB).

```
// Pseudocode minh hoa: Thu thap chi so CPU va RAM
public class MetricsCollector
{
    public MetricSampleDto CollectCpu() =>
        new() { MetricName = "CPU", MetricValue = ReadCpuPercent(), Unit = "%" };

    public MetricSampleDto CollectRam()
    {
        double freeMb = GC.GetGCMemoryInfo().TotalAvailableMemoryBytes / (1024.0
            * 1024.0);
        return new() { MetricName = "RAM", MetricValue = Math.Round(freeMb, 0),
            Unit = "MB" };
    }
}
```

Listing 6: Pseudocode minh họa: Thu thập chỉ số CPU và RAM từ hệ thống

3.2.5 Làm sao thử?

Chạy Agent, mở Task Manager trên máy tính lên so sánh xem con số CPU/RAM do Agent đo được có khớp với Task Manager hay không.

3.2.6 Khi nào coi là xong? (PASS Criteria)

- Giá trị đo CPU nằm trong khoảng hợp lệ từ 0.0% đến 100.0%.
- Giá trị RAM phản ánh đúng dung lượng bộ nhớ khả dụng.
- Đóng gói chính xác vào DTO chuẩn `MetricSampleDto`.

3.3 Chặng 4: VIỆC TIẾP THEO: Gửi nhịp tim định kỳ duy trì sự sống (Heartbeat)

3.3.1 Tại sao cần phần này?

Nếu một máy tính sinh viên bị tắt nguồn đột ngột hoặc rút dây mạng, máy tính đó sẽ không kịp gửi bất kỳ lời chào tạm biệt nào. Gói tin nhịp tim gửi đều đặn mỗi 5 giây giúp máy chủ Central biết máy đó vẫn đang hoạt động bình thường.

3.3.2 Khi chạy, chuyện gì xảy ra?

Cứ đúng mỗi 5 giây, Agent gửi một bản tin `HEARTBEAT` siêu nhẹ (chỉ chứa số thứ tự nhịp tim `sequenceNumber` tăng dần và thời gian hoạt động `uptimeSeconds`). Máy chủ Central nhận được sẽ lập tức trả về `HEARTBEAT_ACK`.

3.3.3 Bạn sẽ đụng file nào?

Tạo mới file: `src/LabGuard.Agent/HeartbeatService.cs`.

3.3.4 Bạn cần làm những bước lớn nào?

1. Dùng `PeriodicTimer` đặt chu kỳ lặp đúng 5 giây.
2. Mỗi nhịp, tăng biến đếm `sequenceNumber` lên 1.
3. Đóng gói bản tin `WireEnvelope` loại `HEARTBEAT` và đẩy vào hàng đợi gửi đi.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Dùng `PeriodicTimer(TimeSpan.FromSeconds(5))` giúp định thời chuẩn xác, không bị trôi thời gian.
- `SequenceNumber` tăng dần liên tục để đối soát thứ tự gói tin.

```
// Pseudocode minh hoa: Dinh ky 5s phat nhip tim
public async Task StartHeartbeatLoopAsync(Action<byte[]> sendAction, string
    deviceId, CancellationToken ct)
{
    using var timer = new PeriodicTimer(TimeSpan.FromSeconds(5));
    long seq = 0;
    while (await timer.WaitForNextTickAsync(ct))
    {
        var payload = new HeartbeatPayload { SequenceNumber = ++seq,
            UptimeSeconds = Environment.TickCount64 / 1000 };
        var env = WireEnvelope<HeartbeatPayload>.Create(MessageTypes.Heartbeat,
            deviceId, payload);
        sendAction(FrameCodec.Encode(env));
    }
}
```

Listing 7: Pseudocode minh họa: Định kỳ 5s gửi nhịp tim sống còn

3.3.5 Làm sao thử?

Theo dõi cửa sổ console của Agent: cứ mỗi 5 giây màn hình phải in log gửi nhịp tim `sequence=1`, `sequence=2` thành công.

3.3.6 Khi nào coi là xong? (PASS Criteria)

- Gửi nhịp tim đều đặn đúng chu kỳ 5 giây (±200ms).
- Số thứ tự `SequenceNumber` tăng đơn điệu liên tục.
- Nhận và xử lý gói tin `HEARTBEAT_ACK` từ Central Server.

3.4 Chặng 5: VIỆC TIẾP THEO: Bộ đệm ngoại tuyến lưu tạm khi rớt mạng (Replay Buffer)

3.4.1 Tại sao cần phần này?

Khi sinh viên vô tình đá phải chân cáp mạng, dây mạng bị lỏng trong chốc lát. Trong khoảng thời gian đó, Agent vẫn đo CPU/RAM nhưng không gửi đi được. Nếu vứt bỏ số liệu thì máy chủ sẽ bị mất dữ liệu lịch sử.

3.4.2 Khi chạy, chuyện gì xảy ra?

Agent lưu các mẻ số liệu đo đạc chưa gửi được vào một hàng đợi trong bộ nhớ RAM (`ReplayBuffer`). Hàng đợi này chứa tối đa 50 mẻ số liệu (`METRIC_BATCH`, tương đương 250 giây dữ liệu), tự động loại bỏ mẻ cũ nhất nếu đầy theo chính sách FIFO Drop-Oldest để không làm tràn RAM. Lưu ý: Chỉ lưu `METRIC_BATCH`, tuyệt đối không lưu `HEARTBEAT` hay `SERVICE_STATUS` vào bộ đệm vì dữ liệu thời gian thực cũ không còn giá trị liveness. Khi kết nối mạng có lại, Agent tự động gửi bù các mẻ này kèm cờ `isReplay = true`.

3.4.3 Bạn sẽ đụng file nào?

Tạo mới file: `src/LabGuard.Agent/ReplayBuffer.cs`.

3.4.4 Bạn cần làm những bước lớn nào?

1. Sử dụng hàng đợi an toàn luồng `ConcurrentQueue<byte[]>`.
2. Chỉ đưa bản tin `METRIC_BATCH` vào bộ đệm; nếu số lượng vượt quá 50 mẻ thì loại bỏ mẻ cũ nhất (`Drop Oldest`).
3. Viết hàm `DrainAsync` để đẩy bù dữ liệu khi mạng phục hồi.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Giới hạn dung lượng 50 mẻ ngăn ngừa lỗi tràn bộ nhớ (OutOfMemoryException), cờ `isReplay = true` thông báo dữ liệu gửi bù.
- Xả hàng đợi theo trật tự Vào trước - Ra trước (FIFO) để bảo đảm dòng thời gian số liệu.

```
// Pseudocode minh hoa: Bo dem hang doi luu tam va phat lai
public class ReplayBuffer
{
    private readonly ConcurrentQueue<byte[]> _queue = new();
    public void Enqueue(byte[] frame)
    {
        if (_queue.Count >= 50) _queue.TryDequeue(out _); // Giu toi da 50 me (250s)
        _queue.Enqueue(frame);
```

```
    }
    public async Task DrainAsync(Func<byte[], Task> sendFunc)
    {
        while (_queue.TryDequeue(out var frame)) await sendFunc(frame);
    }
}
```

Listing 8: Pseudocode minh họa: Bộ đệm hàng đợi lưu tạm và phát lại

3.4.5 Làm sao thử?

Chạy Agent, rút dây mạng ra 10 giây rồi cắm lại: kiểm tra log thấy Agent tự động đẩy bù các gói tin đã gom trong thời gian mất mạng.

3.4.6 Khi nào coi là xong? (PASS Criteria)

- Bộ đệm không bao giờ vượt quá 50 mẻ dữ liệu (chỉ lưu METRIC_BATCH, cờ isReplay=true).
- Bảo toàn thứ tự vào trước ra trước (FIFO).
- Tự động xả sạch bộ đệm ngay khi kết nối TCP được tái thiết lập.

3.5 Chặng 6: VIỆC TIẾP THEO: Giám sát tiến trình phần mềm quy định

3.5.1 Tại sao cần phần này?

Trong phòng thi thực hành, có những phần mềm bắt buộc phải chạy ngầm (ví dụ phần mềm thi cử). Ta cần phát hiện ngay lập tức nếu sinh viên vô tình hoặc cố ý tắt phần mềm này.

3.5.2 Khi chạy, chuyện gì xảy ra?

Agent định kỳ kiểm tra danh sách tiến trình đang hoạt động trên máy tính. Nếu thiếu tiến trình bắt buộc, Agent gửi ngay cảnh báo sự cố lên Central Server.

3.5.3 Bạn sẽ đụng file nào?

Tạo mới file: `src/LabGuard.Agent/ServiceMonitor.cs`.

3.5.4 Bạn cần làm những bước lớn nào?

1. Đọc danh sách tiến trình qua `Process.GetProcesses()`.
2. So khớp tên không phân biệt hoa thường với danh sách bắt buộc.
3. Trả về danh sách các dịch vụ bị thiếu.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- So sánh tên tiến trình không phân biệt hoa thường (`OrdinalIgnoreCase`).
- Trả về danh sách các tiến trình bị tắt để gửi cảnh báo.

```csharp
// Pseudocode minh hoa: Kiem tra tien trinh quy dinh
public class ServiceMonitor
{
    public List<string> CheckMissing(IEnumerable<string> required)
    {
        var running = Process.GetProcesses().Select(p =>
            p.ProcessName).ToHashSet(StringComparer.OrdinalIgnoreCase);
        return required.Where(req => !running.Contains(req)).ToList();
    }
}
```

Listing 9: Pseudocode minh họa: Kiểm tra phần mềm quy định có đang chạy không

3.5.5 Làm sao thử?

Cấu hình theo dõi tiến trình `notepad`, bật Notepad lên rồi đóng lại: Agent lập tức in log cảnh báo phát hiện tiến trình bị tắt.

3.5.6 Khi nào coi là xong? (PASS Criteria)

- Nhận diện chính xác tiến trình bị tắt trong vòng 1 chu kỳ kiểm tra.
- So khớp tên không phân biệt chữ hoa chữ thường.

3.6 Chặng 7: VIỆC TIẾP THEO: Tự động hóa kiểm thử cho Agent

Mã tác vụ nội bộ: LG-A-008, LG-A-009

3.6.1 Tại sao cần phần này?

Viết kiểm thử tự động giúp bạn chắc chắn rằng code Agent của mình chạy ổn định, không bị lỗi bộ nhớ hay lỗi logic trước khi nộp bài cho thầy cô.

3.6.2 Khi chạy, chuyện gì xảy ra?

Lệnh `dotnet test tests/LabGuard.Agent.Tests` sẽ chạy qua các tình huống giả lập: mất mạng, đầy bộ đệm, đọc sai khung tin, và xác nhận kết quả Passed.

3.6.3 Bạn sẽ đụng file nào?

Tạo các tệp kiểm thử trong: `tests/LabGuard.Agent.Tests/`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Bơm 55 mẻ số liệu vào bộ đệm dung lượng 50.
- Khẳng định số lượng phần tử còn lại không bao giờ vượt quá 50 mẻ.

```csharp
// Vi du rut gon: Kiem tra bo dem khong vuot qua 50 me (chi METRIC_BATCH)
[Fact]
public void ReplayBuffer_WhenExceeded_DropsOldest()
{
    var buffer = new ReplayBuffer();
    for (int i = 0; i < 55; i++) buffer.Enqueue(new byte[] { 1, 2, 3 });
```

```
    Assert.True(buffer.Count <= 50);
}
```

Listing 10: Ví dụ rút gọn: Bài kiểm thử đơn vị cho bộ đệm ReplayBuffer

3.6.4 Khi nào coi là xong? (PASS Criteria)

- Lệnh `dotnet test tests/LabGuard.Agent.Tests` chạy hoàn tất thành công.
- 100% các ca kiểm thử đạt kết quả Passed (Failed: 0).

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- 6 chặng của Agent được xây dựng tuần tự: kết nối TCP → đo CPU/RAM → gửi Heartbeat → đệm Replay → giám sát tiến trình → kiểm thử tự động.
- Mọi gói tin đều đi qua `FrameCodec.Encode()` trước khi gửi và kiểm tra an toàn luồng.

4. Năm bẫy lỗi lập trình cần tránh dành cho Thành viên A

Dưới đây là 5 lỗi kỹ thuật phổ biến nhất kèm theo ví dụ so sánh mã nguồn giữa cách viết sai và cách viết chuẩn mực:

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 1: Quên chuyển đổi Big-Endian cho độ dài khung tin
Nếu bạn dùng `BitConverter.GetBytes(length)` trên máy tính chip Intel/AMD (kiến trúc Little-Endian), số nguyên sẽ bị lưu ngược thứ tự byte. Máy chủ đọc vào sẽ ra con số hàng triệu, làm sập chương trình!

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Không dùng `BitConverter` vì phụ thuộc vào chip phần cứng.
- Luôn dùng `BinaryPrimitives.WriteUInt32BigEndian` để bảo đảm chuẩn mạng.

```
// [SAI] Dung BitConverter tren may x86 gay dao nguoc byte:
byte[] prefix = BitConverter.GetBytes(payloadLength); // Nguy hiem!

// [DUNG] Bat buoc dung BinaryPrimitives de dam bao Big-Endian:
byte[] prefix = new byte[4];
BinaryPrimitives.WriteUInt32BigEndian(prefix, (uint)payloadLength); // Chuan mang
```

Listing 11: So sánh Bẫy 1: Little-Endian vs Big-Endian chuẩn

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 2: Tràn bộ nhớ RAM do lưu đệm vô tận khi mất mạng
Nếu mất mạng trong thời gian dài mà Agent cứ nhét gói tin vào bộ nhớ không giới hạn, chương trình sẽ ngốn sạch RAM máy tính sinh viên (*Out Of Memory*). Luôn luôn đặt giới hạn trần cho bộ nhớ đệm (ví dụ 50 mẻ METRIC_BATCH) và loại bỏ mẻ cũ nhất khi đầy.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Dùng hàng đợi không giới hạn sẽ làm cạn kiệt RAM khi rớt mạng lâu.
- Luôn có điều kiện kiểm tra `Count >= 50` và loại bỏ phần tử cũ nhất.

```
// [SAI] Hang doi khong gioi han lam tran bo nho RAM:
_unboundedQueue.Enqueue(packet); // Co the ngon hang Gigabyte RAM neu mat mang lau!

// [DUNG] Bounded Queue co co che go bo ban tin cu nhat:
while (_boundedQueue.Count >= 50) _boundedQueue.TryDequeue(out _);
_boundedQueue.Enqueue(packet);
```

Listing 12: So sánh Bẫy 2: Hàng đợi vô hạn vs Bounded FIFO Queue

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 3: Đọc Socket bằng ReadAsync mà không đọc lặp
Một lần gọi `ReadAsync` không bảo đảm trả về đủ khung tin. Nếu bạn lấy luôn dữ liệu chưa đủ byte để giải mã JSON, hệ thống sẽ ném ngoại lệ `JsonException`. Luôn dùng vòng lặp tích lũy byte như trong `FrameCodec.ReadExactlyBytesAsync`.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 4: Thử kết nối lại socket quá nhanh trong vòng lặp vô tận
Nếu dùng vòng lặp `while(true)` gọi `ConnectAsync` liên tục không có độ trễ nghỉ (`Task.Delay`), CPU máy tính sẽ bị ngốn 100% ngay lập tức. Luôn dùng thuật toán Exponential Backoff.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Vòng lặp bận không nghỉ sẽ đốt cháy 100% CPU.
- Tăng dần thời gian nghỉ theo cấp số nhân (1s, 2s, 4s . . . 30s) giúp tiết kiệm tài nguyên.

```
// [SAI] Vong lap ban (Busy-Waiting) dot chay 100% CPU:
while (!connected) { Connect(); } // Khong co delay!

// [DUNG] Exponential Backoff tang dan thoi gian nghi:
await Task.Delay(TimeSpan.FromSeconds(delaySec), token);
delaySec = Math.Min(delaySec * 2, 30);
```

Listing 13: So sánh Bẫy 4: Vòng lặp bận (Busy Wait) vs Exponential Backoff

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 5: Sai lệch định dạng thời gian UTC
Khi gán trường `sentAtUtc`, nếu bạn dùng `DateTime.Now.ToString()` thì thời gian sẽ mang múi giờ địa phương (+7) và thiếu ký tự `Z`. Luôn dùng `DateTime.UtcNow.ToString("O")`.

5. Bộ câu hỏi vấn đáp dành riêng cho Thành viên A

Dưới đây là 20 câu hỏi trọng tâm kèm câu trả lời chi tiết giúp Thành viên A tự tin đạt điểm xuất sắc khi bảo vệ đồ án trước hội đồng:

Câu 1: Tại sao em lại chọn kích thước khung tin tối đa là 262,144 byte (256 KiB)?
Trả lời: Giới hạn 256 KiB (262,144 byte) là hoàn toàn dư dả cho các gói tin nhịp tim và báo cáo tài nguyên phần cứng (thường chỉ chiếm từ 200 byte đến vài chục kilobyte). Việc đặt giới hạn cứng này giúp máy chủ phòng vệ hiệu quả trước các cuộc tấn công từ chối dịch vụ (DoS), ngăn chặn việc cấp phát mảng byte khổng lồ làm tràn bộ nhớ RAM.

Câu 2: Tại sao độ dài khung tin phải dùng định dạng Big-Endian?
Trả lời: Big-Endian là chuẩn Network Byte Order theo quy chuẩn RFC 1700 của mạng Internet, trong đó byte có trọng số lớn nhất (MSB) được gửi trước. Điều này bảo đảm tính tương thích tuyệt đối giữa các máy tính sử dụng kiến trúc CPU khác nhau (x86 dùng Little-Endian, ARM/MIPS có thể dùng Big-Endian).

Câu 3: WireEnvelope của em có mấy trường? Kể tên?
Trả lời: Có chính xác 6 trường cố định: `version` (int = 1), `messageType` (string), `requestId` (string GUID), `deviceId` (string), `sentAtUtc` (string ISO 8601 kết thúc bằng 'Z'), và `payload` (JsonElement).

Câu 4: Tại sao version lại là số nguyên 1 mà không phải chuỗi "1.0"?
Trả lời: Sử dụng kiểu số nguyên `int` giúp CPU so sánh giá trị nhanh hơn chỉ với 1 lệnh máy, tiết kiệm băng thông mạng và tránh được các lỗi phân tích chuỗi ký tự giữa các ngôn ngữ lập trình khác nhau.

Câu 5: Tại sao Agent lại là Client mà không phải Server?
Trả lời: Agent chạy trên máy tính sinh viên, thường nằm sau tường lửa Windows Firewall và mạng NAT. Tường lửa mặc định chặn kết nối đi vào (Inbound) nhưng cho phép kết nối đi ra (Outbound). Do đó Agent phải đóng vai trò Client chủ động mở kết nối ra ngoài (Active Open) tới địa chỉ IP và cổng tĩnh của Central Server.

Câu 6: Bản tin nhịp tim gửi bao lâu một lần?
Trả lời: Bản tin `HEARTBEAT` được gửi định kỳ chính xác mỗi 5 giây một lần.

Câu 7: Tên chuẩn của thông điệp nhịp tim là gì?
Trả lời: Tên chuẩn là `HEARTBEAT` (chiều Agent → Central) và `HEARTBEAT_ACK` (chiều Central → Agent).

Câu 8: Hiện tượng phân mảnh trong TCP là gì và em giải quyết như thế nào?
Trả lời: TCP là giao thức hướng dòng byte không có ranh giới tin nhắn. Một gói tin gửi đi có thể bị chia nhỏ thành nhiều mảnh khi truyền qua các thiết bị mạng. Em giải quyết triệt để bằng phương thức `ReadExactlyBytesAsync`: sử dụng vòng lặp `while` đọc tích lũy từng mẩu dữ liệu cho đến khi gom đủ chính xác số byte ghi trong tiêu đề 4 byte mới xử lý tiếp.

Câu 9: Thuật toán Exponential Backoff hoạt động ra sao khi mất mạng?
Trả lời: Khi kết nối bị đứt, Agent chờ 1 giây trước khi thử lại. Nếu tiếp tục thất bại, thời gian chờ tăng gấp đôi: 2s, 4s, 8s, 16s và cố định ở mức trần 30s. Thuật toán này giúp giảm tải cho mạng và máy chủ khi có sự cố diện rộng.

Câu 10: Bộ nhớ đệm Replay Buffer giải quyết vấn đề gì?
Trả lời: Giúp duy trì tính liên tục của dữ liệu lịch sử. Khi sinh viên vô tình làm lỏng cáp

mạng, các số liệu CPU/RAM vẫn được ghi nhớ trong hàng đợi của Agent và tự động phát lại tuần tự khi kết nối mạng được phục hồi.

Câu 11: Chính sách đẩy dữ liệu của Replay Buffer là gì?
Trả lời: Sử dụng cơ chế hàng đợi FIFO (First-In, First-Out), gửi từ bản ghi cũ nhất đến bản ghi mới nhất nhằm bảo đảm trình tự dòng thời gian trong cơ sở dữ liệu.

Câu 12: Điều gì xảy ra nếu bộ đệm ReplayBuffer bị đầy 50 mẻ dữ liệu?
Trả lời: Hệ thống áp dụng chính sách loại bỏ mẻ cũ nhất (`FIFO Drop-Oldest`) để nhường chỗ cho mẻ mới nhất, bảo đảm ứng dụng không bao giờ vượt quá ngưỡng tiêu thụ RAM cho phép. Lưu ý rằng chỉ mẻ số liệu (`METRIC_BATCH`) được lưu giữ kèm cờ `isReplay = true`, tuyệt đối không lưu `HEARTBEAT` hay `SERVICE_STATUS`.

Câu 13: Em đo phần trăm CPU trên Windows bằng cách nào trong C#?
Trả lời: Bằng cách lấy hiệu số thời gian thực thi của tiến trình (`Process.TotalProcessorTime`) chia cho tích của số lượng nhân CPU (`Environment.ProcessorCount`) và khoảng thời gian trôi qua giữa 2 lần lấy mẫu.

Câu 14: Tại sao sentAtUtc bắt buộc phải có chữ Z ở cuối?
Trả lời: Ký tự 'Z' đại diện cho giờ chuẩn quốc tế UTC (Zulu Time), loại bỏ hoàn toàn nguy cơ sai lệch thời gian khi máy trạm và máy chủ đặt ở các múi giờ khác nhau.

Câu 15: Nếu máy tính sinh viên đổi tên thì Agent nhận biết thế nào?
Trả lời: Agent đọc tên máy trực tiếp từ biến hệ thống `Environment.MachineName` tại thời điểm khởi tạo và gán vào trường `deviceId` trong mọi phong bì gửi đi.

Câu 16: Tại sao FrameCodec lại ném lỗi khi độ dài payload bằng 0?
Trả lời: Vì một khung tin có chiều dài bằng 0 không mang nội dung nghiệp vụ, có thể xuất phát từ việc lập trình sai hoặc cố tình gửi gói tin rác làm nghẽn máy chủ.

Câu 17: Làm thế nào để tắt tiến trình Agent một cách an toàn (Graceful Shutdown)?
Trả lời: Bằng cách lắng nghe sự kiện `Console.CancelKeyPress` hoặc tín hiệu hủy từ `CancellationTokenSource`, gửi bản tin `AGENT_GOODBYE` lên máy chủ và đóng kết nối `NetworkStream` trước khi tiến trình kết thúc.

Câu 18: Trong thư viện Protocol, 107 bài test kiểm tra những gì?
Trả lời: 107 bài test kiểm thử bao phủ toàn diện: đóng khung nhị phân Big-Endian (35 bài), kiểm tra tính hợp lệ của WireEnvelope (28 bài), đối soát 16 fixture JSON chuẩn cùng các DTO (24 bài), và bộ xác thực EnvelopeValidator (20 bài).

Câu 19: Thành viên B và C sẽ dùng code của em như thế nào?
Trả lời: Các bạn thêm tham chiếu dự án `<ProjectReference Include="...LabGuard.Protocol.cs />` vào dự án Central và Admin để dùng chung `FrameCodec`, `WireEnvelope` và các DTO.

Câu 20: Nếu sinh viên cố tình rút cáp mạng, Agent có biết ngay lập tức không?
Trả lời: Khi rút cáp vật lý, hệ điều hành không gửi tín hiệu TCP ngắt kết nối. Tuy nhiên, khi Agent đến chu kỳ gửi gói tin tiếp theo, phương thức `WriteAsync` sẽ ném ngoại lệ `SocketException`, giúp Agent lập tức phát hiện mất mạng và chuyển sang chế độ lưu đệm.

— HẾT CẨM NANG HƯỚNG DẪN THÀNH VIÊN A —